

ФЕДЕРАЛЬНОЕ АГЕНТСТВО ПО ОБРАЗОВАНИЮ

ГОСУДАРСТВЕННОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО
ПРОФЕССИОНАЛЬНОГО ОБРАЗОВАНИЯ «САМАРСКИЙ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ имени
академика С.П. КОРОЛЁВА»

КАФЕДРА «ТЕХНИЧЕСКАЯ КИБЕРНЕТИКА»

ОТЧЕТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ

«Объектно-ориентированное программирование»

ЛАБОРАТОРНАЯ РАБОТА №3

Студент _____ Ветров И.Р.

Группа _____ 6301-030301D

Руководитель _____ Борисов Д. С.

Оценка _____

Задание 2

Было добавлено исключение для индексов с использованием try-catch в Main через getMessage() позволяющая вывести причину исключения.

```
1 package functions;
2
3 public class FunctionPointIndexOutOfBoundsException extends IndexOutOfBoundsException {
4     public FunctionPointIndexOutOfBoundsException() {
5         super(s: "Выход за границы набора точек функции");
6     }
7 }
```

Также было добавлено исключение для остальных случаев.

```
1 package functions;
2
3 public class InappropriateFunctionPointException extends Exception {
4
5     public InappropriateFunctionPointException() {
6         super(message: "Некорректная операция с точкой функции");
7     }
8 }
```

Задание 3

Были добавлены в каждом геттер и сеттер методе выбрасывания исключения FunctionPointIndexOutOfBoundsException. Также InappropriateFunctionPointException, IllegalStateException были добавлены в deletePoint, addPoint.

```
public ArrayTabulatedFunction(double leftX, double rightX, int pointsCount)
    throws IllegalArgumentException {
    if (leftX >= rightX) {
        throw new IllegalArgumentException("Левая граница (" + leftX +
            ") должна быть меньше правой (" + rightX + ")");
    }

    if (pointsCount < 2) {
        throw new IllegalArgumentException("Количество точек должно быть не менее 2, получено: " + pointsCount);
    }
}
```

```
public ArrayTabulatedFunction(double leftX, double rightX, double[] values)
    throws IllegalArgumentException {
    if (leftX >= rightX) {
        throw new IllegalArgumentException("Левая граница (" + leftX +
            ") должна быть меньше правой (" + rightX + ")");
    }

    if (values.length < 2) {
        throw new IllegalArgumentException("Количество точек должно быть не менее 2, получено: " + values.length);
    }
}
```

```

public FunctionPoint getPoint(int index) throws FunctionPointIndexOutOfBoundsException {
    if (index < 0 || index >= pointsCount) {
        throw new FunctionPointIndexOutOfBoundsException(index, minIndex: 0, pointsCount - 1);
    }
    return new FunctionPoint(points[index]);
}

public void setPoint(int index, FunctionPoint point)
    throws FunctionPointIndexOutOfBoundsException, InappropriateFunctionPointException {
    if (index < 0 || index >= pointsCount) {
        throw new FunctionPointIndexOutOfBoundsException(index, minIndex: 0, pointsCount - 1);
    }

    if (index > 0 && point.getX() <= points[index - 1].getX() + EPSILON) {
        throw new InappropriateFunctionPointException(
            point.getX(), points[index - 1].getX(), points[index + 1].getX());
    }

    if (index < pointsCount - 1 && point.getX() >= points[index + 1].getX() - EPSILON) {
        throw new InappropriateFunctionPointException(
            point.getX(), points[index - 1].getX(), points[index + 1].getX());
    }
}

```

```

public double getPointY(int index) throws FunctionPointIndexOutOfBoundsException {
    if (index < 0 || index >= pointsCount) {
        throw new FunctionPointIndexOutOfBoundsException(index, minIndex: 0, pointsCount - 1);
    }
    return points[index].getY();
}

public void setPointY(int index, double y) throws FunctionPointIndexOutOfBoundsException {
    if (index < 0 || index >= pointsCount) {
        throw new FunctionPointIndexOutOfBoundsException(index, minIndex: 0, pointsCount - 1);
    }
    points[index].setY(y);
}

public void deletePoint(int index)
    throws FunctionPointIndexOutOfBoundsException, IllegalStateException {
    if (index < 0 || index >= pointsCount) {
        throw new FunctionPointIndexOutOfBoundsException(index, minIndex: 0, pointsCount - 1);
    }

    if (pointsCount <= 2) {
        throw new IllegalStateException(s: "Невозможно удалить точку: минимальное количество точек - 2");
    }
}

```

Задание 4

Был создан класс для работы со связным списком, имеющий в себе конструктор, позволяющий создать новый узел в списке, методы позволяющие получить или изменить точку в узле.

```

public class LinkedListTabulatedFunction implements TabulatedFunction {
    private class FunctionNode {
        private FunctionPoint point;
        private FunctionNode prev;
        private FunctionNode next;

        public FunctionNode(FunctionPoint point) {
            this.point = point;
        }

        public FunctionPoint getPoint() {
            return point;
        }

        public void setPoint(FunctionPoint point) {
            this.point = point;
        }

        public FunctionNode getPrev() {
            return prev;
        }

        public void setPrev(FunctionNode prev) {
            this.prev = prev;
        }

        public FunctionNode getNext() {
            return next;
        }

        public void setNext(FunctionNode next) {
            this.next = next;
        }
    }
}

```

Метод позволяющий получить точку находящуюся на переданном индексе, для оптимизации поиска.

```

private FunctionNode getNodeByIndex(int index) throws FunctionPointIndexOutOfBoundsException {
    if (index < 0 || index >= pointsCount) {
        throw new FunctionPointIndexOutOfBoundsException(index, minIndex: 0, pointsCount - 1);
    }

    if (pointsCount == 0) {
        return head;
    }

    if (lastNode != null && lastNode != head) {
        int distanceFromLast = Math.abs(index - lastIndex);

        if (index == lastIndex) {
            return lastNode;
        }

        if (distanceFromLast <= index && distanceFromLast <= pointsCount - 1 - index) {
            return moveFromNode(lastNode, lastIndex, index);
        }
    }

    if (index <= pointsCount / 2) {
        lastNode = moveFromNode(head.getNext(), startIndex: 0, index);
    } else {
        lastNode = moveFromNode(head.getPrev(), pointsCount - 1, index);
    }

    lastIndex = index;
    return lastNode;
}

```

Метод позволяющий добавить элемент в конец списка.

```

private FunctionNode addNodeToTail() {
    FunctionNode newNode = new FunctionNode(point: null);

    FunctionNode tail = head.getPrev();

    newNode.setPrev(tail);
    newNode.setNext(head);

    tail.setNext(newNode);
    head.setPrev(newNode);

    pointsCount++;
    return newNode;
}

```

Метод позволяющий добавить элемент на переданный индекс.

```

private FunctionNode addNodeByIndex(int index) throws FunctionPointIndexOutOfBoundsException {
    if (index < 0 || index > pointsCount) {
        throw new FunctionPointIndexOutOfBoundsException(index, minIndex: 0, pointsCount);
    }

    if (index == pointsCount) {
        return addNodeToTail();
    }

    FunctionNode nodeAtIndex = getNodeByIndex(index);
    FunctionNode newNode = new FunctionNode(point: null);

    FunctionNode prevNode = nodeAtIndex.getPrev();
    newNode.setPrev(prevNode);
    newNode.setNext(nodeAtIndex);

    prevNode.setNext(newNode);
    nodeAtIndex.setPrev(newNode);

    pointsCount++;

    if (index <= lastIndex) {
        lastIndex++;
    }

    return newNode;
}

```

Метод позволяющий удалить точку по переданному индексу.

```

private FunctionNode deleteNodeByIndex(int index) throws FunctionPointIndexOutOfBoundsException {
    if (index < 0 || index >= pointsCount) {
        throw new FunctionPointIndexOutOfBoundsException(index, minIndex: 0, pointsCount - 1);
    }

    FunctionNode nodeToDelete = getNodeByIndex(index);

    FunctionNode prevNode = nodeToDelete.getPrev();
    FunctionNode nextNode = nodeToDelete.getNext();

    prevNode.setNext(nextNode);
    nextNode.setPrev(prevNode);

    nodeToDelete.setPrev(prev: null);
    nodeToDelete.setNext(next: null);

    pointsCount--;

    if (nodeToDelete == lastNode) {
        lastNode = nextNode;
        if (lastNode == head) {
            if (pointsCount > 0) {
                lastNode = head.getNext();
                lastIndex = 0;
            } else {
                lastNode = head;
                lastIndex = -1;
            }
        }
    } else if (index < lastIndex) {
        lastIndex--;
    }

    return nodeToDelete;
}

```


Два конструктора создающие список.

```
public LinkedListTabulatedFunction(double leftX, double rightX, int pointsCount)
    throws IllegalArgumentException {
    if (leftX >= rightX) {
        throw new IllegalArgumentException("Левая граница (" + leftX +
            " | " + rightX + ") должна быть меньше правой (" + rightX + ")");
    }

    if (pointsCount < 2) {
        throw new IllegalArgumentException("Количество точек должно быть не менее 2, получено: " + pointsCount);
    }

    initList();
    this.pointsCount = pointsCount;

    double step = (rightX - leftX) / (pointsCount - 1);
    for (int i = 0; i < pointsCount; i++) {
        double x = leftX + i * step;
        FunctionNode newNode = addNodeToTail();
        newNode.setPoint(new FunctionPoint(x, y: 0));
    }

    initCache();
}
```

```
public LinkedListTabulatedFunction(double leftX, double rightX, double[] values)
    throws IllegalArgumentException {
    if (leftX >= rightX) {
        throw new IllegalArgumentException("Левая граница (" + leftX +
            " | " + rightX + ") должна быть меньше правой (" + rightX + ")");
    }

    if (values.length < 2) {
        throw new IllegalArgumentException("Количество точек должно быть не менее 2, получено: " + values.length);
    }

    initList();
    this.pointsCount = values.length;

    double step = (rightX - leftX) / (values.length - 1);
    for (int i = 0; i < values.length; i++) {
        double x = leftX + i * step;
        FunctionNode newNode = addNodeToTail();
        newNode.setPoint(new FunctionPoint(x, values[i]));
    }

    initCache();
}
```

Методы для левой и правой границы списка.

```
public double getLeftDomainBorder() {
    if (pointsCount == 0) {
        return Double.NaN;
    }
    return head.getNext().getPoint().getX();
}

public double getRightDomainBorder() {
    if (pointsCount == 0) {
        return Double.NaN;
    }
    return head.getPrev().getPoint().getX();
}
```

Метод нахождения координаты Y с помощью линейной интерполяции для переданного X.

```

public double getFunctionValue(double x) {
    if (pointsCount == 0) {
        return Double.NaN;
    }

    double leftBorder = getLeftDomainBorder();
    double rightBorder = getRightDomainBorder();

    if (x < leftBorder - EPSILON || x > rightBorder + EPSILON) {
        return Double.NaN;
    }

    FunctionNode currentNode = head.getNext();
    while (currentNode != head) {
        FunctionNode nextNode = currentNode.getNext();
        if (nextNode == head) break;

        double x1 = currentNode.getPoint().getX();
        double x2 = nextNode.getPoint().getX();

        if (x >= x1 - EPSILON && x <= x2 + EPSILON) {
            if (Math.abs(x - x1) < EPSILON) {
                return currentNode.getPoint().getY();
            }
            if (Math.abs(x - x2) < EPSILON) {
                return nextNode.getPoint().getY();
            }

            double y1 = currentNode.getPoint().getY();
            double y2 = nextNode.getPoint().getY();
            return y1 + (y2 - y1) * (x - x1) / (x2 - x1);
        }

        currentNode = nextNode;
    }

    return Double.NaN;
}

```

Методы для получения количества точек в списке и для получения и изменения точки в списке.


```

public int getPointsCount() {
    return pointsCount;
}

public FunctionPoint getPoint(int index) throws FunctionPointIndexOutOfBoundsException {
    FunctionNode node = getNodeByIndex(index);
    return new FunctionPoint(node.getPoint());
}

public void setPoint(int index, FunctionPoint point)
    throws FunctionPointIndexOutOfBoundsException, InappropriateFunctionPointException {

    FunctionNode node = getNodeByIndex(index);

    double newX = point.getX();
    double nodeX = node.getPoint().getX();

    if (Math.abs(newX - nodeX) < EPSILON) {
        node.getPoint().setY(point.getY());
        return;
    }

    FunctionNode prevNode = node.getPrev();
    FunctionNode nextNode = node.getNext();

    if (prevNode != head) {
        double prevX = prevNode.getPoint().getX();
        if (newX <= prevX + EPSILON) {
            throw new InappropriateFunctionPointException(
                "Новая координата X (" + newX + ") должна быть больше предыдущей (" + prevX + ")");
        }
    }

    if (nextNode != head) {
        double nextX = nextNode.getPoint().getX();
        if (newX >= nextX - EPSILON) {
            throw new InappropriateFunctionPointException(
                "Новая координата X (" + newX + ") должна быть меньше следующей (" + nextX + ")");
        }
    }

    node.getPoint().setX(newX);
    node.getPoint().setY(point.getY());
}

```

Геттер и сеттор методы.

```

public double getPointX(int index) throws FunctionPointIndexOutOfBoundsException {
    return getNodeByIndex(index).getPoint().getX();
}

public void setPointX(int index, double x)
    throws FunctionPointIndexOutOfBoundsException, InappropriateFunctionPointException {
    FunctionNode node = getNodeByIndex(index);

    double currentX = node.getPoint().getX();
    if (Math.abs(x - currentX) < EPSILON) {
        return; // X не изменился
    }

    FunctionNode prevNode = node.getPrev();
    FunctionNode nextNode = node.getNext();

    if (prevNode != head) {
        double prevX = prevNode.getPoint().getX();
        if (x <= prevX + EPSILON) {
            throw new InappropriateFunctionPointException(
                "Новая координата X (" + x + ") должна быть больше предыдущей (" + prevX + ")");
        }
    }

    if (nextNode != head) {
        double nextX = nextNode.getPoint().getX();
        if (x >= nextX - EPSILON) {
            throw new InappropriateFunctionPointException(
                "Новая координата X (" + x + ") должна быть меньше следующей (" + nextX + ")");
        }
    }

    node.getPoint().setX(x);
}

```

Метод удаление точек.

```

public void deletePoint(int index)
    throws FunctionPointIndexOutOfBoundsException, IllegalStateException {
    if (pointsCount <= 2) {
        throw new IllegalStateException(s: "Невозможно удалить точку: минимальное количество точек - 2");
    }

    deleteNodeByIndex(index);
}

```

Метод добавления точек.

```

public void addPoint(FunctionPoint point) throws InappropriateFunctionPointException {
    if (pointsCount == 0) {
        FunctionNode newNode = addNodeToTail();
        newNode.setPoint(new FunctionPoint(point));
        initCache();
        return;
    }

    double newX = point.getX();

    double leftBorder = getLeftDomainBorder();
    double rightBorder = getRightDomainBorder();

    if (newX < leftBorder - EPSILON) {
        // Добавляем в начало
        FunctionNode newNode = addNodeByIndex(index: 0);
        newNode.setPoint(new FunctionPoint(point));
        return;
    }

    if (newX > rightBorder + EPSILON) {
        FunctionNode newNode = addNodeToTail();
        newNode.setPoint(new FunctionPoint(point));
        return;
    }

    int insertIndex = 0;
    FunctionNode currentNode = head.getNext();

    while (currentNode != head && newX > currentNode.getPoint().getX() + EPSILON) {
        currentNode = currentNode.getNext();
        insertIndex++;
    }

    if (currentNode != head &&
        Math.abs(newX - currentNode.getPoint().getX()) < EPSILON) {
        throw new InappropriateFunctionPointException(
            "Точка с координатой X = " + newX + " уже существует");
    }

    FunctionNode newNode = addNodeByIndex(insertIndex);
    newNode.setPoint(new FunctionPoint(point));
}

```

Задание 6

Был создан интерфейс, благодаря которому программа понимает с каким классом необходимо работать.

```

1 package functions;
2
3 public interface TabulatedFunction {
4     double getLeftDomainBorder();
5     double getRightDomainBorder();
6     double getFunctionValue(double x);
7
8     int getPointsCount();
9     FunctionPoint getPoint(int index) throws FunctionPointIndexOutOfBoundsException;
10    void setPoint(int index, FunctionPoint point)
11        | throws FunctionPointIndexOutOfBoundsException, InappropriateFunctionPointException;
12    double getPointX(int index) throws FunctionPointIndexOutOfBoundsException;
13    void setPointX(int index, double x)
14        | throws FunctionPointIndexOutOfBoundsException, InappropriateFunctionPointException;
15    double getPointY(int index) throws FunctionPointIndexOutOfBoundsException;
16    void setPointY(int index, double y) throws FunctionPointIndexOutOfBoundsException;
17
18    void deletePoint(int index)
19        | throws FunctionPointIndexOutOfBoundsException, IllegalStateException;
20    void addPoint(FunctionPoint point) throws InappropriateFunctionPointException;
21
22    void printPoints();
23 }

```

Задание 7

Была добавлена проверка исключений.

```
private static void testArrayFunction() {
    System.out.println(x: "1. Тестирование ArrayTabulatedFunction:");

    try {
        TabulatedFunction parabola = new ArrayTabulatedFunction(-2, rightX: 2, pointsCount: 5);

        for (int i = 0; i < parabola.getPointsCount(); i++) {
            double x = parabola.getPointX(i);
            parabola.setPointY(i, x * x);
        }

        parabola.printPoints();

        // Тестируем вычисления
        System.out.println(x: "\n    Вычисления:");
        System.out.printf(format: "    f(0.5) = %.4f\n", parabola.getFunctionValue(x: 0.5));
        System.out.printf(format: "    f(3) = %s\n", parabola.getFunctionValue(x: 3));

        // Добавляем точку
        parabola.addPoint(new FunctionPoint(x: 1.5, y: 2.25));
        System.out.println(x: "\n    После добавления точки (1.5, 2.25):");
        parabola.printPoints(); // БЫЛО printPoint()

    } catch (Exception e) {
        System.out.println("    Ошибка: " + e.getMessage());
    }
}
```

```

private static void testLinkedListFunction() {
    System.out.println(x: "2. Тестирование LinkedListTabulatedFunction:");

    try {
        double[] sinValues = {0, 0.707, 1, 0.707, 0, -0.707, -1, -0.707, 0};
        TabulatedFunction sinFunc = new LinkedListTabulatedFunction(leftX: 0, Math.PI * 2, sinValues);

        sinFunc.printPoints();

        System.out.println(x: "\n    Вычисления:");
        System.out.printf(format: "    f(%.2f) = %.4f\n", Math.PI/4, sinFunc.getFunctionValue(Math.PI/4));
        System.out.printf(format: "    f(%.2f) = %.4f\n", Math.PI, sinFunc.getFunctionValue(Math.PI));

        sinFunc.deletePoint(index: 3);
        System.out.println(x: "\n    После удаления точки с индексом 3:");
        sinFunc.printPoints();
    } catch (Exception e) {
        System.out.println("    Ошибка: " + e.getMessage());
    }
}

```

```

0 private static void testExceptions() {
    System.out.println(x: "3. Тестирование исключений:");

    System.out.println(x: "\n    a) Некорректные параметры конструктора:");
    try {
        TabulatedFunction badFunc = new ArrayTabulatedFunction(leftX: 10, rightX: 5, pointsCount: 5);
    } catch (IllegalArgumentException e) {
        System.out.println("    Поймано: " + e.getClass().getSimpleName() + " - " + e.getMessage());
    }

    System.out.println(x: "\n    b) Выход за границы массива:");
    try {
        TabulatedFunction func = new ArrayTabulatedFunction(leftX: 0, rightX: 10, pointsCount: 5);
        func.getPoint(index: 10);
    } catch (FunctionPointIndexOutOfBoundsException e) {
        System.out.println("    Поймано: " + e.getClass().getSimpleName() + " - " + e.getMessage());
    }

    System.out.println(x: "\n    c) Нарушение порядка точек:");
    try {
        TabulatedFunction func = new ArrayTabulatedFunction(leftX: 0, rightX: 10, pointsCount: 5);
        func.setPoint(index: 2, new FunctionPoint(x: 8, y: 0)); // Должно быть между 5 и 7.5
    } catch (InappropriateFunctionPointException e) {
        System.out.println("    Поймано: " + e.getClass().getSimpleName() + " - " + e.getMessage());
    }

    System.out.println(x: "\n    d) Удаление при недостаточном количестве точек:");
    try {
        TabulatedFunction func = new ArrayTabulatedFunction(leftX: 0, rightX: 10, pointsCount: 3);
        func.deletePoint(index: 0);
        func.deletePoint(index: 0); // Осталась 1 точка
    } catch (IllegalStateException e) {
        System.out.println("    Поймано: " + e.getClass().getSimpleName() + " - " + e.getMessage());
    }

    System.out.println(x: "\n    e) Добавление точки с существующим X:");
    try {
        TabulatedFunction func = new ArrayTabulatedFunction(leftX: 0, rightX: 10, pointsCount: 3);
        func.addPoint(new FunctionPoint(x: 5, y: 100));
    } catch (InappropriateFunctionPointException e) {
        System.out.println("    Поймано: " + e.getClass().getSimpleName() + " - " + e.getMessage());
    }
}

```

```

    System.out.println(x: "\n    f) Тест с LinkedList:");
    try {
        TabulatedFunction linkedFunc = new LinkedListTabulatedFunction(leftX: 0, rightX: 5, pointsCount: 4);
        System.out.println(x: "    LinkedList функция создана успешно");
        linkedFunc.printPoints();
    } catch (Exception e) {
        System.out.println("    Ошибка: " + e.getMessage());
    }
}

```


Результаты работы программы

=== Тестирование Табулированных функций ===

1. Тестирование ArrayTabulatedFunction:

Табулированная функция (5 точек):

[0]: (-2,00, 4,00)
[1]: (-1,00, 1,00)
[2]: (0,00, 0,00)
[3]: (1,00, 1,00)
[4]: (2,00, 4,00)

Вычисления:

$f(0.5) = 0,5000$

$f(3) = \text{NaN}$

После добавления точки (1.5, 2.25):

Табулированная функция (6 точек):

[0]: (-2,00, 4,00)
[1]: (-1,00, 1,00)
[2]: (0,00, 0,00)
[3]: (1,00, 1,00)
[4]: (1,50, 2,25)
[5]: (2,00, 4,00)

=====

2. Тестирование LinkedListTabulatedFunction:

Табулированная функция (связный список, 18 точек):

[0]: (0,0000, 0,0000)
[1]: (0,7854, 0,7070)
[2]: (1,5708, 1,0000)
[3]: (2,3562, 0,7070)
[4]: (3,1416, 0,0000)
[5]: (3,9270, -0,7070)
[6]: (4,7124, -1,0000)
[7]: (5,4978, -0,7070)
[8]: (6,2832, 0,0000)

Вычисления:

$f(0,79) = 0,7070$

$f(3,14) = 0,0000$

После удаления точки с индексом 3:

Табулированная функция (связный список, 17 точек):

[0]: (0,0000, 0,0000)
[1]: (0,7854, 0,7070)
[2]: (1,5708, 1,0000)
[3]: (3,1416, 0,0000)
[4]: (3,9270, -0,7070)
[5]: (4,7124, -1,0000)
[6]: (5,4978, -0,7070)
[7]: (6,2832, 0,0000)

=====

=====

3. Тестирование исключений:

а) Некорректные параметры конструктора:

Поймано: `IllegalArgumentException` - Левая граница (10.0) должна быть меньше правой (5.0)

б) Выход за границы массива:

Поймано: `FunctionPointIndexOutOfBoundsException` - Индекс 10 выходит за допустимый диапазон [0, 4]

с) Нарушение порядка точек:

Поймано: `InappropriateFunctionPointException` - Координата X = 8.0 должна находиться в интервале (2.5, 7.5)

д) Удаление при недостаточном количестве точек:

Поймано: `IllegalStateException` - Невозможно удалить точку: минимальное количество точек - 2

е) Добавление точки с существующим X:

Поймано: `InappropriateFunctionPointException` - Точка с координатой X = 5.0 уже существует в функции

ф) Тест с `LinkedList`:

`LinkedList` функция создана успешно

Табулированная функция (связный список, 8 точек):

[0]: (0,0000, 0,0000)

[1]: (1,6667, 0,0000)

[2]: (3,3333, 0,0000)

[3]: (5,0000, 0,0000)

PS D:\lab3>